PROJECT PROPOSAL
Implementation of a DNS Server

Technologies: BIND9 • Linux • Python

1. Project Title, Group Numbers, Names & IDs

Project Title	**Implementation of a DNS Server with Caching and Security Features**
Group Member 1	Muhammad Ali | Roll No: 23K-0052
Group Member 2	Muhammad Abser | Roll No: 23K-0030

2. Proposed Project Description

This project aims to implement a fully functional, local DNS (Domain Name System) server using BIND9 on a Linux environment, enhanced with a Python-based layer for caching management and security enforcement. The system will serve as a recursive and/or authoritative DNS resolver capable of resolving domain names to IP addresses efficiently while protecting against common DNS-based attacks.

How It Works

The DNS server will be built on a Linux (Ubuntu) machine using BIND9 as the core resolver engine. Queries from client machines will be intercepted by the BIND9 daemon, which will attempt to resolve them either from its local zone files (for authoritative queries) or by recursively querying upstream root and TLD servers. A Python module, leveraging the dnspython and scapy libraries, will sit alongside BIND9 to manage the caching layer and apply security rules before and after resolution.

Functional Features

- **Resolve both forward (domain → IP) and reverse (IP → domain) DNS queries using BIND9 configured with appropriate zone files.**DNS Query Resolution:
- **Implement an in-memory DNS cache to store previously resolved query responses. Each cached entry respects its TTL (Time-To-Live) value; expired entries are evicted automatically to ensure freshness.**Caching with TTL Management:
- **Apply 0x20 encoding (randomised capitalisation of query names) and source port randomisation to outgoing queries, making it significantly harder for attackers to inject forged responses.**Cache Poisoning Prevention:

- **Restrict the rate at which a single source IP can issue DNS queries to the server, preventing DNS amplification and flood-based denial-of-service attacks.**Query Rate Limiting:
- **Enable DNSSEC (DNS Security Extensions) in BIND9 to cryptographically validate responses from signed zones, ensuring data integrity and authenticity.**DNSSEC Validation:
- **Measure and compare average query resolution times under cached vs. non-cached conditions to quantify the performance gains of the caching layer.**Performance Benchmarking:
- **Maintain detailed query logs (source IP, queried domain, response time, cache hit/miss) for analysis and security auditing.**Logging & Monitoring:

3. Plan of Work

The project will be completed over 5 weeks. The following table outlines the key milestones:

Week	Milestone / Task
Week 1	Environment setup: Install Ubuntu/Linux VM, BIND9, Python libraries (dnspython, scapy). Define system architecture and DNS zone file structure.
Week 2	Core DNS resolver: Configure BIND9 as a recursive/authoritative resolver. Implement forward/reverse lookups and basic DNS query handling in Python.
Week 3	Caching layer: Implement DNS response caching with TTL management. Measure and compare query resolution time with and without cache.
Week 4	Security module: Implement DNSSEC validation, cache poisoning prevention (0x20 encoding, source port randomisation), and query rate limiting.
Week 5	Integration, testing & reporting: End-to-end integration testing, performance benchmarks, documentation, and final report preparation.

a. Individual Contributions

The team is divided as follows, with each member responsible for distinct functional features:

Muhammad Ali (23K-0052)

- **Functional Feature 1** — DNS Query Resolution
  - Configure BIND9 zone files for forward and reverse lookup zones.

- Set up the recursive resolver to handle iterative queries to root/TLD servers.

- **Functional Feature 2** — Caching with TTL Management
  - Implement the Python caching module using dnspython to intercept and store responses.
  - Develop TTL-aware eviction logic to keep the cache current.
  - Conduct performance benchmarks comparing cached vs. non-cached resolution times.
- **Functional Feature 7** — Logging & Monitoring
  - Design and implement the logging framework to capture query metadata and cache hit/miss statistics.

Muhammad Abser (23K-0030)

- **Functional Feature 3** — Cache Poisoning Prevention
  - Implement 0x20 encoding for outgoing DNS queries using Python/scapy.
  - Enforce source port randomisation and response validation rules.
- **Functional Feature 4** — Query Rate Limiting
  - Develop rate-limiting logic in Python to throttle excessive requests per source IP.
  - Integrate rate-limiting rules with BIND9's response policy zones (RPZ) or as a front-end filter.
- **Functional Feature 5** — DNSSEC Validation
  - Enable and configure DNSSEC in BIND9 (trust anchors, key management).
  - Test validation against signed and unsigned zones to verify correct enforcement.

4. References

- ISC BIND 9 Administrator Reference Manual. Internet Systems Consortium. Available at https://bind9.readthedocs.io
- Albitz, P. & Liu, C. (2006). DNS and BIND (5th ed.). O'Reilly Media.
- dnspython Documentation. Available at: https://www.dnspython.org/docs/
- Scapy Project Documentation. Available at: https://scapy.readthedocs.io
- Arends, R. et al. (2005). RFC 4033 – DNS Security Introduction and Requirements. IETF.
- Vixie, P. & Schryver, V. (2012). RFC 5358 – Preventing Use of Recursive Nameservers in Reflector Attacks. IETF.
- Kaminsky, D. (2008). It's the End of the Cache As We Know It. Black Hat USA Presentation.
- Ubuntu Server Documentation. DNS Configuration. Available at: https://ubuntu.com/server/docs/service-domain-name-service-dns

End of Proposal